AISSMS
INSTITUTE OF INFORMATION TECHNOLOGY
ADDING VALUE TO ENGINEERING

Department of Computer Engineering

A Web Technology Mini Project Report

"TaskFlow: A Full-Stack Task and Productivity Management Web Application"

SUBMITTED TO THE DEPARTMENT OF COMPUTER ENGINEERING

AISSMS IOIT

T. Y. BTech Engineering

SUBMITTED BY

Prathmesh Tulshiram Adhav (Roll No. 01)

PRN: 24410143

2026-27



AISSMS
INSTITUTE OF INFORMATION TECHNOLOGY
ADDING VALUE TO ENGINEERING

All India Shri Shivaji Memorial Society's Institute of Information Technology, Pune-01

(An Autonomous Institute)

Affiliated to Savitribai Phule Pune University, Pune.

Department of Computer Engineering

CERTIFICATE

This is to certify that the project report

"TaskFlow: A Full-Stack Task and Productivity Management Web Application"

Submitted by

Prathmesh Tulshiram Adhav (Roll No. 01, PRN: 24410143) is a bonafide student of this institute and the work has been carried out by him/her under the supervision of Ms. Prajakta Kamble and it is approved for the partial fulfillment of the Department of Computer Engineering AISSMS IOIT.



Ms. Prajakta Kamble
Guide
Department of Computer Engineering

Dr. K.S.Wagh
Head,
Department of Computer Engineering

Place : Pune

Date :

AISSMS
INSTITUTE OF INFORMATION TECHNOLOGY
ADDING VALUE TO ENGINEERING

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to **Ms. Prajakta Kamble** for her valuable guidance, continuous support, and encouragement throughout the development of this Web Technology mini project. Her guidance and suggestions played an important role in the successful completion of this project.

I would also like to extend my sincere thanks to **AISSMS Institute of Information Technology, Pune**, and the Department of Computer Engineering for providing the necessary support and learning environment

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of this mini project.

Prathmesh Tulshiram Adhav
Roll No.: 01

ABSTRACT

TaskFlow is a full-stack task and productivity management web application developed as a Web Technology mini project. Users register, sign in securely and manage tasks with priorities and optional due dates. The dashboard provides statistics, a completion-rate bar, overdue detection, and real-time search, filtering and sorting. A Next.js 16 frontend (TypeScript, React 19) communicates over REST with a layered Go `net/http` backend that stores data in MySQL 8.0.

1.1 Context

Students, professionals and small teams track many assignments, deadlines and goals, and web-based task managers make this information available from any browser. TaskFlow is designed as a focused, distraction-free tool that combines a modern component-based user interface with a secure, layered REST API and a relational database.

1.2 Problem Statement

Tasks maintained on paper, in scattered notes or in spreadsheets are difficult to prioritise, search and review, and they provide no automatic indication of overdue work or overall progress. A multi-user web application must also keep each user's data private and resist attacks such as SQL injection, XSS-based token theft and CSRF. The problem addressed is to design and implement a secure web application that stores each user's tasks in a structured database and presents them through an interactive, responsive interface.

1.3 Objective

- To implement secure registration and session-based authentication with bcrypt and HttpOnly cookies.
- To provide the complete task lifecycle (create, read, update, delete, status toggle) with priorities, due dates and overdue detection.
- To provide search, filtering, sorting, statistics and a normalised MySQL schema behind a layered Go REST API with per-user data isolation.

INDEX

Sr. No.	Chapter No	Title Of Chapter		Page no.
1	**1**	**ABSTRACT**	**……**	**4**
	1.1	Context	……	4
	1.2	Problem Statement	……	4
	1.3	Objective	……	4
2	**2**	**INTRODUCTION**	**……**	**6**
	2.1	Background Information on the Chosen Topic	……	6
		Importance of Databases in the Task Management Domain	……	6
	2.2	Scope of the Project	……	7
	2.3	Functional and Non-Functional Requirements	……	8
	2.4	System Requirements	……	8
3	**3**	**SYSTEM DESIGN**	**……**	**9**
	3.1	Database Schema and UML Diagrams	……	9
	3.2	Tables and Relationships	……	16
	3.3	Application / User Interface Design	……	18
4	**4**	**IMPLEMENTATION**	**……**	**19**
	4.1	Description of the Development Environment	……	19
	4.2	Tools and Technologies Used	……	19
	4.3	Backend Implementation	……	20
	4.4	REST API Implementation	……	21
	4.5	Frontend Implementation	……	22
	4.6	Security Implementation	……	23
	4.7	Setup and Execution	……	23
5	**5**	**OUTPUT**	**……**	**24**
	5.1	Public and Authentication Screens	……	24
	5.2	Dashboard and Task Management Screens	……	25
6	**6**	**ADVANTAGES & LIMITATIONS**	**……**	**28**
	6.1	Advantages	……	28
	6.2	Limitations	……	28
	6.3	Future Enhancements	……	29
7	**7**	**CONCLUSION**	**……**	**30**
	7.1	Summary of the Project Achievements	……	30
	7.2	Key Learning	……	30
8	**8**	**REFERENCES**	**……**	**31**
	8.1	Books, Articles, Websites and Other Resources	……	31

INTRODUCTION

2.1 Background Information on the Chosen Topic

Modern web applications are commonly built as two cooperating parts: a client-side application that renders the user interface, and a server-side API that applies business rules and persists data, exchanging JSON through RESTful endpoints over HTTP.

TaskFlow applies this model to personal task management. Its frontend is built with Next.js 16 using the App Router and the Turbopack bundler, React 19 and TypeScript, styled with Tailwind CSS and a custom design system. Interactive screens are implemented as Client Components (marked with `"use client"`) that fetch data asynchronously from the backend. The backend is written in Go and uses the standard library `net/http` server together with the `httprouter` request multiplexer, while MySQL 8.0 stores users, sessions and tasks. The project thus covers core Web Technology topics: component-based HTML rendering, CSS, client-side validation, HTTP methods and status codes, cookies and sessions, CORS, REST API design and database connectivity.

Importance of Databases in the Task Management Domain

A task manager is fundamentally a data-driven application. Every dashboard element is computed from stored records. A relational database such as MySQL provides persistent storage that survives server restarts, enforces data integrity through primary keys, foreign keys, unique and check constraints, and supports efficient filtered and sorted retrieval through indexes. In TaskFlow the InnoDB storage engine supplies ACID transactions and foreign-key enforcement, the `utf8mb4` character set with `utf8mb4_unicode_ci` collation allows full Unicode (including emoji) in task titles and descriptions, and composite indexes beginning with `user_id` match the dashboard's most frequent per-user queries.

Existing System and Proposed System

In the existing approach, tasks are commonly recorded on paper, in personal notes or in spreadsheets. Such methods offer no access control, require manual effort to sort and prioritise, cannot highlight overdue items automatically and provide no summary of progress.

The proposed system, TaskFlow, replaces these methods with an authenticated web application. Each user's tasks are stored in a central database and are visible only to that user, because every task query is scoped by the authenticated user's ID. The dashboard computes statistics automatically, marks pending tasks whose due date has passed as overdue, and allows instant searching, filtering and sorting. A responsive interface with toast notifications and modal forms makes these operations quick to perform.

2.2 Scope of the Project

The scope of TaskFlow is a single-user-per-account task management application intended for individuals and teams who want focused task tracking with priorities, deadlines and progress statistics. The following functionality is within the documented scope:

- Account registration, login, logout and retrieval of the current user's profile.
- Creation, listing, retrieval, editing, deletion and completion toggling of tasks owned by the signed-in user.
- Task attributes: title (1 to 200 characters), optional description, status (pending or completed), priority (low, medium or high) and optional due date.
- Search by title or description, filtering by status and priority, six sort orders, and limit/offset pagination (default 50, maximum 100 items).
- Dashboard statistics: total, pending, completed, high-priority and overdue counts and the completion rate.
- Local deployment: MySQL (3306), Go API (8080) and Next.js frontend (3000).

Shared task lists between users, role-based administration, notifications and production deployment configuration are not part of the documented implementation and are discussed as possible future enhancements in Chapter 6.

2.3 Functional and Non-Functional Requirements

Table 2.1: Functional requirements of TaskFlow

ID	Functional requirement	Supporting endpoint / screen
FR-1	A visitor can register with name, email and password.	`POST /auth/register`, /register
FR-2	A registered user can log in and receive a session cookie.	`POST /auth/login`, /login
FR-3	The signed-in user's profile is shown in the dashboard.	`GET /auth/me`
FR-4	The user can log out, which clears the session cookie.	`POST /auth/logout`
FR-5	The user can create a task with title, description, priority and due date.	`POST /tasks`, task modal
FR-6	The user can list, search, filter, sort and paginate tasks.	`GET /tasks`, filter bar
FR-7	The user can view and edit a single task.	`GET` / `PUT /tasks/:id`
FR-8	The user can toggle a task between pending and completed.	`PATCH /tasks/:id/status`
FR-9	The user can delete a task after confirmation.	`DELETE /tasks/:id`
FR-10	The dashboard shows statistics and the completion rate.	`GET /tasks/stats`

Table 2.2: Non-functional requirements

Quality attribute	Requirement as addressed by the design
Security	bcrypt password hashes, SHA-256 hashed session tokens, HttpOnly and SameSite=Lax cookies, parameterised SQL, exact-origin CORS
Data isolation	every task query restricted by the authenticated `user_id`
Integrity	foreign keys with cascade delete, unique email and token hash, check constraints
Performance	per-user composite indexes, `database/sql` connection pooling, HTTP server timeouts
Usability	responsive design system, toast feedback, modal task editor
Maintainability	monorepo, layered backend, strict TypeScript, ordered idempotent migrations

2.4 System Requirements

The specification documents the software stack rather than minimum hardware figures. The required software is: Go 1.22 or later (developed with Go 1.27 on Windows amd64); Node.js with npm for Next.js 16.4.3, React 19 and TypeScript; MySQL Server 8.0 or later; and a modern

browser with JavaScript and cookies enabled. Ports 3000 (frontend), 8080 (API) and 3306 (MySQL) must be available.

SYSTEM DESIGN

System design describes how TaskFlow is organised before and during implementation: which components exist, how they communicate, what data is stored and how users interact with the system. This chapter presents the overall architecture, the UML and data-flow diagrams, the database schema with its tables and relationships, and the design of the user interface.

3.1 Database Schema and UML Diagrams

3.1.1 System Architecture

TaskFlow follows a decoupled, layered client-server architecture, shown in Figure 3.1. The browser loads the Next.js frontend (port 3000), which calls the Go REST API (port 8080, base URL `/api/v1`) with JSON bodies and credentialed requests so that the session cookie is included. Inside the API, requests pass through the router and middleware layer, the handler layer and the repository layer before reaching the MySQL database (port 3306). Responses travel back along the same path as JSON objects wrapped in a `data` field.

The backend follows a clean layered (repository) pattern. The **router** (`internal/router`) registers routes with httprouter's radix-tree multiplexer and wraps them with CORS and authentication middleware. The **handlers** (`internal/handlers`) form the controller layer: they decode requests, validate input, manage cookies, call the repository and encode responses. The **repository** (`internal/repository`) is the only layer that executes SQL, always through parameterised queries. The **models** (`internal/models`) define the `User`, `Task` and `Stats` structures with JSON tags, and **config** (`internal/config`) loads environment variables, automatically reading the `.env` file. This separation keeps HTTP concerns, business rules and data access independent, so each layer can be understood and changed in isolation.

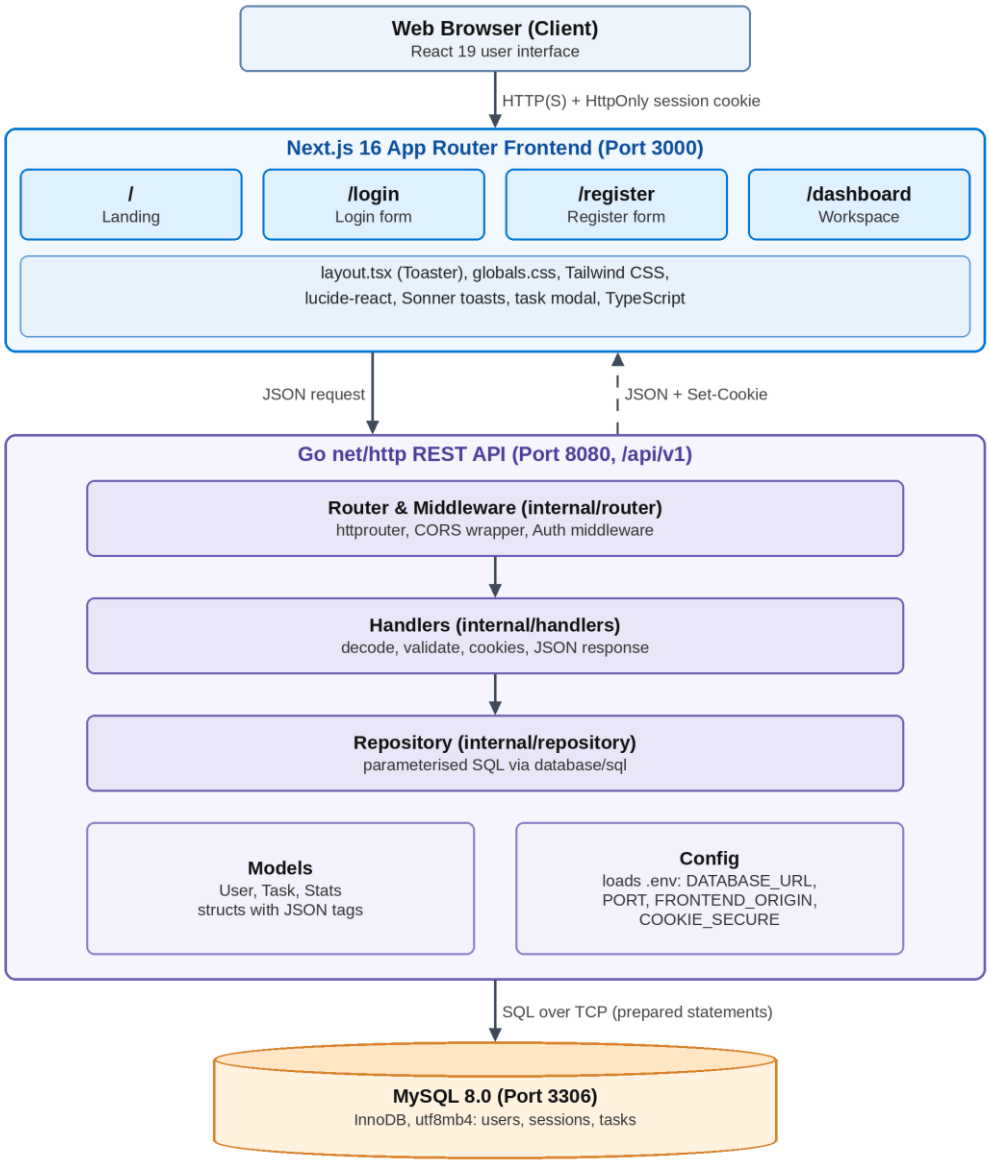


Figure 3.1: System architecture of TaskFlow

3.1.2 Frontend Architecture

Figure 3.2 shows the structure of the Next.js App Router frontend. The root layout (`app/layout.tsx`) wraps every route and provides the Sonner toast provider, while `app/globals.css` defines design tokens, cards, badges and the modal. The four routes are the landing page (/), the login and registration pages and the dashboard. Successful login or

registration redirects to `/dashboard`, whose components call the REST endpoints listed at the bottom of the figure.

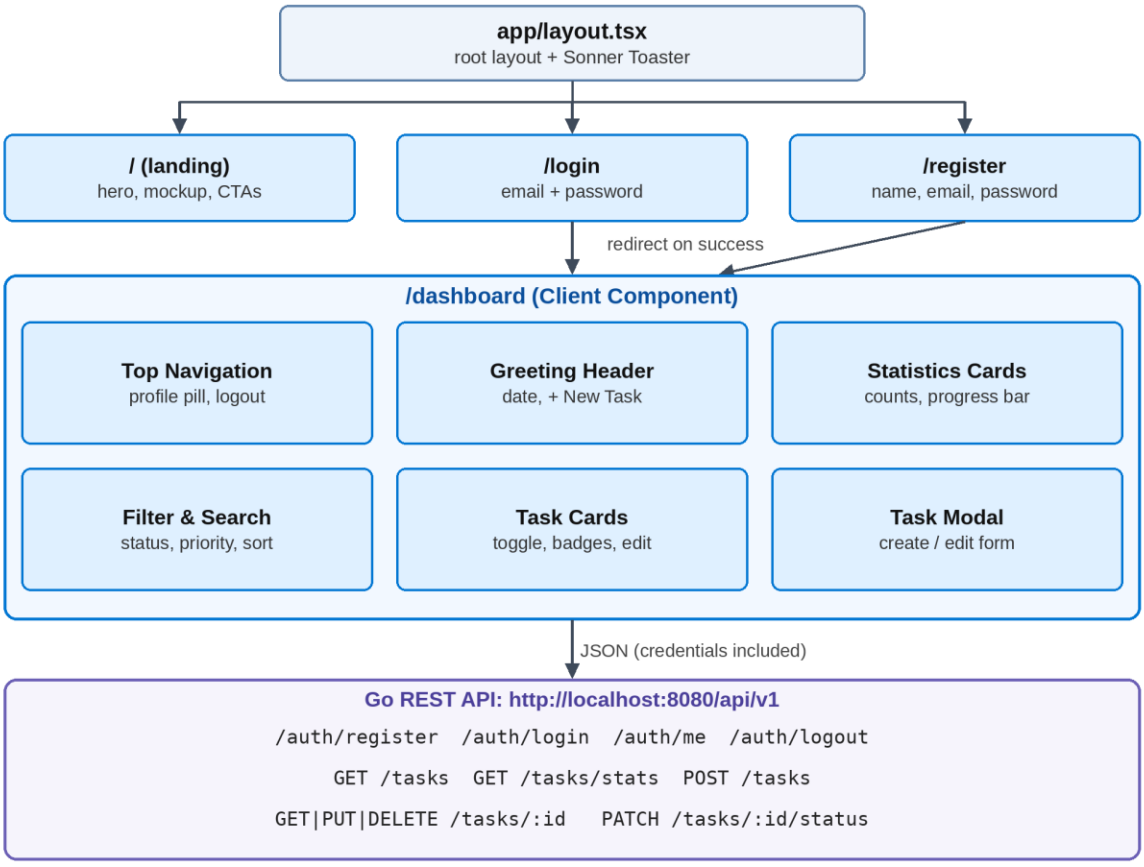


Figure 3.2: Frontend routes, dashboard components and API communication

3.1.3 Use Case Diagram

The use case diagram in Figure 3.3 identifies two actors. A **Visitor** can view the landing page, register and log in. A **Registered User**, a specialisation of the visitor, can manage tasks, search and filter them, view statistics and the profile, and log out. Every use case of the registered user includes *Authenticate Session*, because the corresponding endpoints require a valid session cookie.

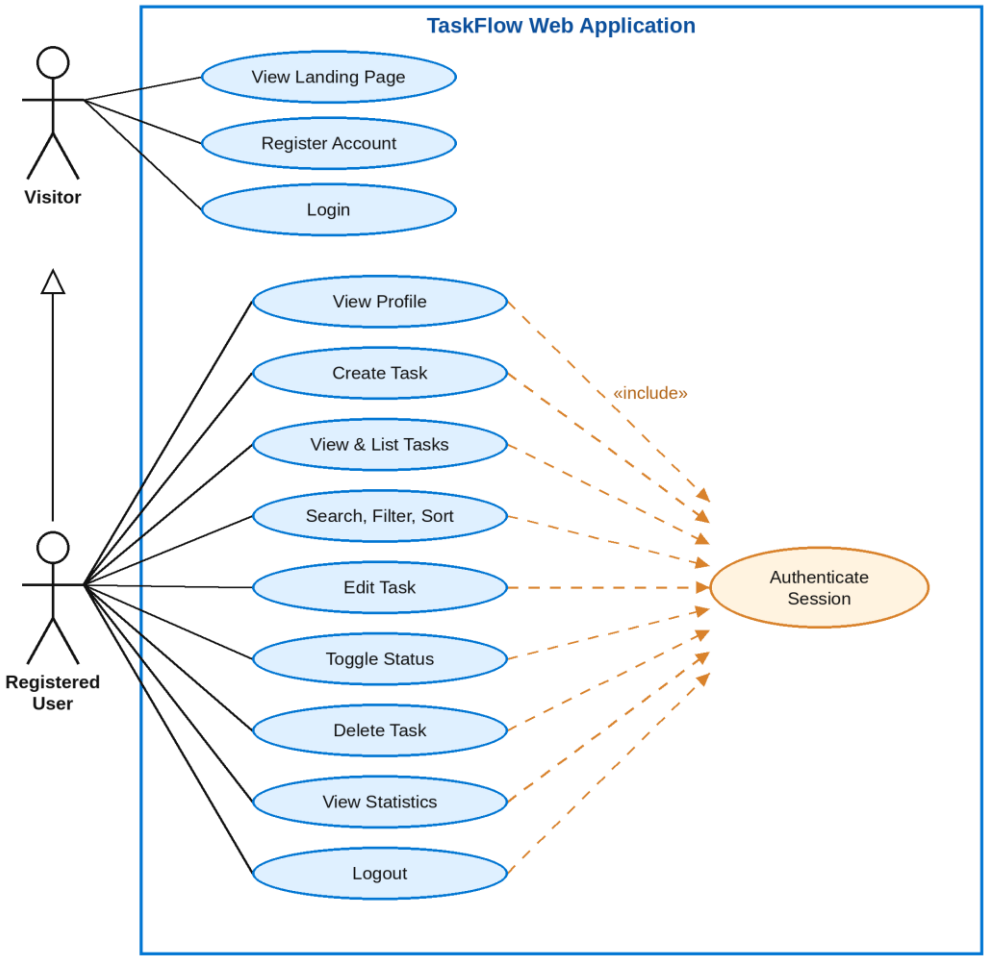


Figure 3.3: UML use case diagram of TaskFlow

3.1.4 Entity Relationship Diagram

Figure 3.4 shows the relational schema. The `users` entity is the parent of two child entities. One user may have zero or more sessions and may own zero or more tasks; each session and each task belongs to exactly one user through its `user_id` foreign key. Both foreign keys use `ON DELETE CASCADE`, so deleting a user also removes that user's sessions and tasks.

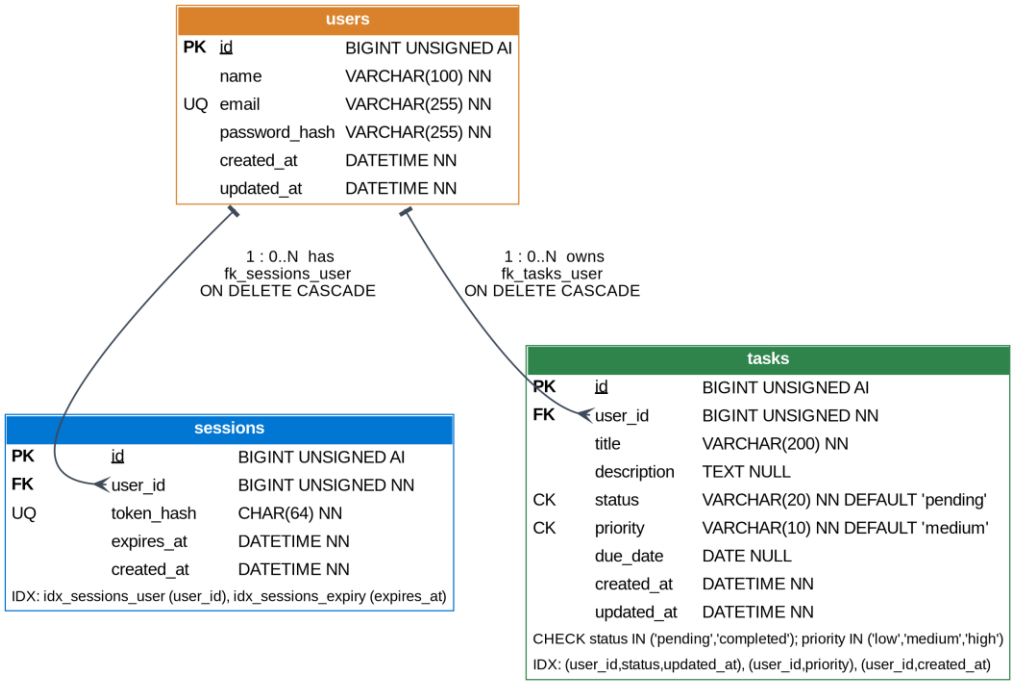


Figure 3.4: Entity relationship diagram (PK, FK, UQ, CK and indexes)

3.1.5 Sequence Diagrams

Figure 3.5 illustrates the login workflow. After client-side validation, the login page sends the credentials to `POST /auth/login`. The handler retrieves the user by email, verifies the password against the stored bcrypt hash, generates a 32-byte random token with `crypto/rand`, stores only its SHA-256 hash in the `sessions` table and returns the raw token in the HttpOnly `taskflow_session` cookie. The frontend then redirects to the dashboard.

Figure 3.6 illustrates task creation. The browser automatically sends the session cookie with the request; the authentication middleware hashes the token, resolves the session to a user ID and passes the request to the task handler, which validates the input and asks the repository to insert the task for that user. The API responds with `201 Created` and the new task object.

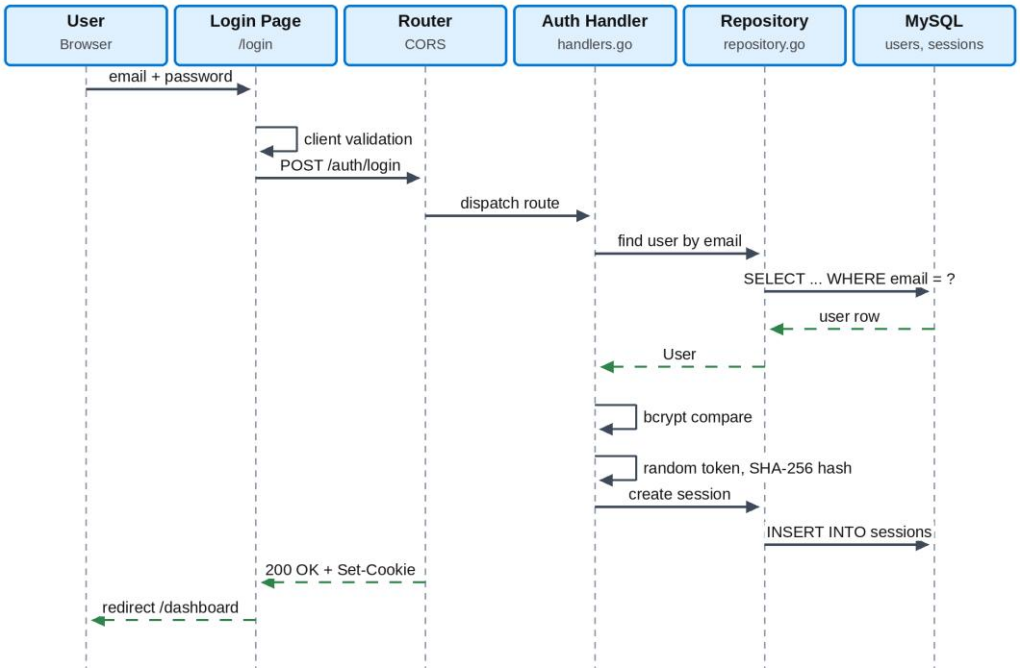


Figure 3.5: Sequence diagram for user login

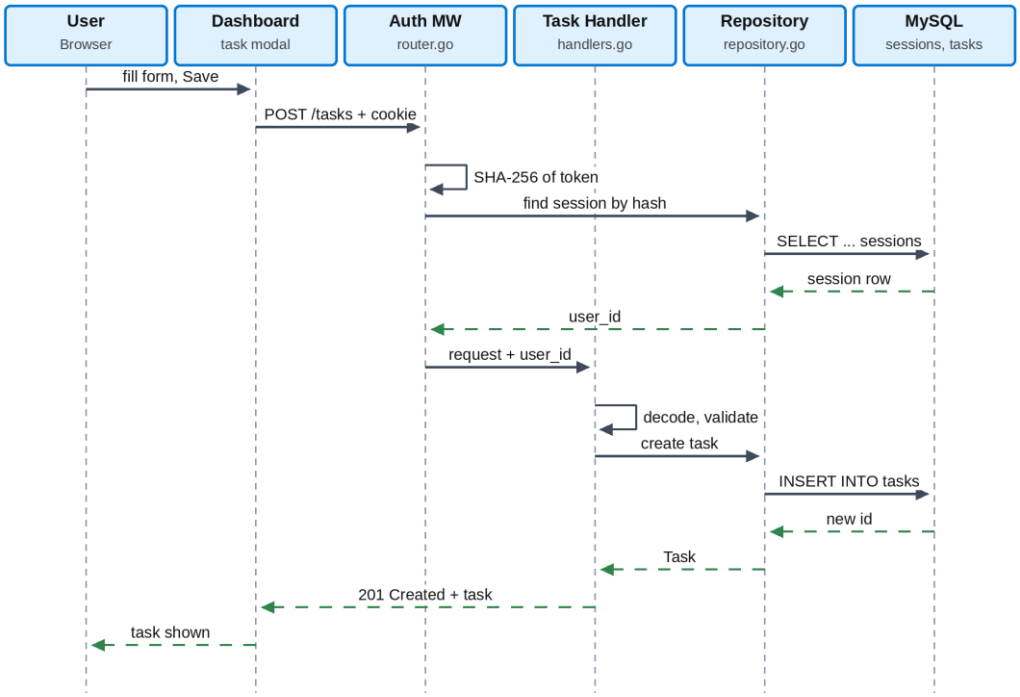


Figure 3.6: Sequence diagram for creating a task

3.1.6 Data Flow Diagrams

The context-level DFD (Figure 3.7) treats TaskFlow as a single process that exchanges data with one external entity, the user. The level-1 DFD (Figure 3.8) decomposes it into five processes and three data stores corresponding to the database tables.

In the level-1 diagram, process 1.0 creates users and sessions, process 2.0 validates the session on each protected request and supplies the user ID, process 3.0 performs task changes on D3, process 4.0 reads matching tasks for the search, filter and sort criteria, and process 5.0 aggregates the user's tasks into the statistics shown on the dashboard.

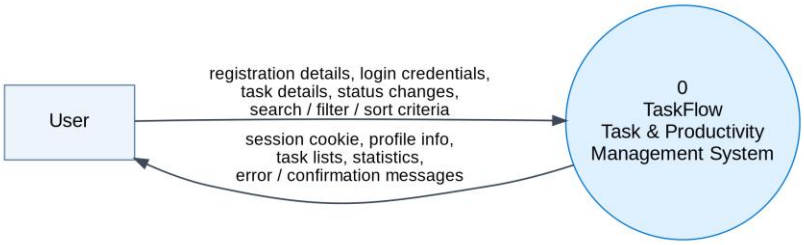


Figure 3.7: Context-level (level-0) data flow diagram

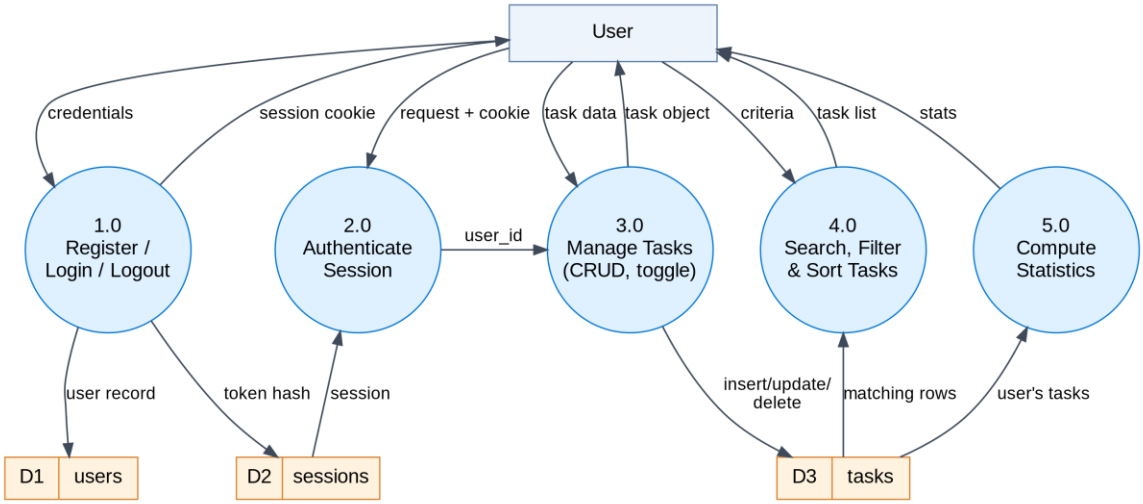


Figure 3.8: Level-1 data flow diagram

3.2 Tables and Relationships

The database `taskflow` contains three InnoDB tables using the `utf8mb4` character set and `utf8mb4_unicode_ci` collation. They are created by numbered migration scripts: `001_create_users.sql`, `002_create_sessions.sql`, `003_create_tasks.sql` and `004_add_priority_to_tasks.sql`, the last of which adds the `priority` column to installations created before priorities existed. The data dictionaries below are taken from the documented schema; where no default is listed, none is specified.

Table 3.1: Data dictionary of the users table

Column	Data type	Key	Null	Constraints / default	Description
id	BIGINT UNSIGNED	PK	No	AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)		No		Full name of the user
email	VARCHAR(255)	UQ	No	UNIQUE	Login email address
password_hash	VARCHAR(255)		No	bcrypt hash	Salted password hash; plaintext never stored
created_at	DATETIME		No	UTC timestamp	Account creation time
updated_at	DATETIME		No	UTC timestamp	Last update time

Table 3.2: Data dictionary of the sessions table

Column	Data type	Key	Null	Constraints / default	Description
id	BIGINT UNSIGNED	PK	No	AUTO_INCREMENT	Unique session identifier
user_id	BIGINT UNSIGNED	FK	No	REFERENCES users(id) ON DELETE CASCADE	Owner of the session
token_hash	CHAR(64)	UQ	No	UNIQUE	Hex-encoded SHA-256 hash of the session token
expires_at	DATETIME		No		Session expiry date and time
created_at	DATETIME		No	UTC timestamp	Session creation time

Table 3.3: Data dictionary of the tasks table

Column	Data type	Key	Null	Constraints / default	Description
id	BIGINT UNSIGNED	PK	No	AUTO_INCREMENT	Unique task identifier
user_id	BIGINT UNSIGNED	FK	No	REFERENCES users(id) ON DELETE CASCADE	Owner of the task
title	VARCHAR(200)		No	1 to 200 characters	Task title

Column	Data type	Key	Null	Constraints / default	Description
description	TEXT		Yes		Optional notes or links
status	VARCHAR(20)		No	DEFAULT 'pending'; CHECK IN ('pending','completed')	Completion state
priority	VARCHAR(10)		No	DEFAULT 'medium'; CHECK IN ('low','medium','high')	Task priority
due_date	DATE		Yes	YYYY-MM-DD	Optional target date
created_at	DATETIME		No	UTC timestamp	Creation time
updated_at	DATETIME		No	UTC timestamp	Last update time

Table 3.4: Indexes and constraints

Table	Index / constraint	Columns	Purpose
sessions	`idx_sessions_user`	(user_id)	Fast session-to-user resolution
sessions	`idx_sessions_expiry`	(expires_at)	Efficient cleanup of expired sessions
sessions	`fk_sessions_user`	user_id → users(id)	Referential integrity, cascade delete
tasks	`idx_tasks_user_status_updated`	(user_id, status, updated_at)	Dashboard queries by status
tasks	`idx_tasks_user_priority`	(user_id, priority)	Priority filters
tasks	`idx_tasks_user_created`	(user_id, created_at)	Default chronological sorting
tasks	`chk_tasks_status`, `chk_tasks_priority`	status, priority	Allow only documented values
tasks	`fk_tasks_user`	user_id → users(id)	Referential integrity, cascade delete

Relationships and integrity

Both relationships are one-to-many: `users` (1) to `sessions` (0..N) and `users` (1) to `tasks` (0..N). The foreign keys guarantee that no session or task can reference a non-existent user, and `ON DELETE CASCADE` keeps the database consistent when a user row is deleted. Primary keys are surrogate `BIGINT UNSIGNED AUTO_INCREMENT` values, the unique constraint on `email` prevents duplicate accounts, the unique constraint on `token_hash` ensures every session token maps to exactly one session, and the two check constraints prevent invalid status or priority values even if application validation were bypassed.

Normalisation, indexing and transactions

Each table stores facts about a single entity, every non-key column depends on the whole primary key, and no column repeats information held in another table: user details are stored once in `users` and referenced by ID. The schema therefore satisfies third normal form. The composite indexes all begin with `user_id` because every task query is scoped to the authenticated user; MySQL can then use the leftmost prefix of an index to filter by user and status or priority and to return rows in creation order. Statistics such as overdue counts are computed from the stored `status` and `due_date` values rather than stored separately, which avoids redundant data. InnoDB provides ACID transactions and row-level locking, so each insert, update or delete statement executes atomically; the specification does not document explicit multi-statement transactions.

3.3 Application / User Interface Design

The interface follows a custom responsive design system built with Tailwind CSS, CSS variables, Inter typography and gradients, with lucide-react icons and Sonner toast notifications. The **landing page** presents the brand, a hero headline and value proposition, a live interactive mockup preview and call-to-action buttons leading to login and registration. The **authentication pages** validate input on the client, show error toasts and redirect to the dashboard on success.

The **dashboard** is arranged from top to bottom: a navigation bar with the logo, a profile pill (gradient avatar with initials, name and email) and a logout button; a greeting header with the weekday, date and + *New Task* button; four statistics cards (total, pending with the high-priority count, completed, and completion rate with a progress bar); and a filter bar with status pills, priority and sort dropdowns and a search input with a clear action. Each **task card** offers a circular completion toggle, a title that is struck through when completed, the optional description, a colour-coded priority badge (soft red with a flame icon for High, amber for Medium, blue for Low), a due-date chip that turns into a red *Overdue* badge for pending tasks past their date, and edit and delete buttons. The **create/edit modal** contains a required title, an optional description, three priority buttons and a native date picker.

IMPLEMENTATION

4.1 Description of the Development Environment

TaskFlow is developed as a monorepo containing two independent applications, `backend/` (the Go module `taskflow/backend`) and `frontend/` (the Next.js client), together with the project README and specification. The backend was developed and run with Go 1.27 on Windows amd64 against a local MySQL 8.0 server. The directory structure is shown below.

```
project/
|-- README.md, PROJECT_DETAILS.txt
|-- backend/                        Go REST API (module taskflow/backend)
|   |-- .env, .env.example          environment variables (.env is gitignored)
|   |-- go.mod, go.sum              module definition and dependency checksums
|   |-- cmd/api/main.go             API server entry point
|   |-- cmd/migrate/main.go         idempotent migration runner
|   |-- internal/config/            config loader with .env parser
|   |-- internal/handlers/          HTTP handlers, validation, cookies
|   |-- internal/models/            User, Task, Stats structs with JSON tags
|   |-- internal/repository/        database operations, auto-migration hooks
|   |-- internal/router/            route registry, CORS wrapper, auth middleware
|   `-- migrations/001..004_*.sql SQL migrations run in numerical order
`-- frontend/                       Next.js App Router client
    |-- package.json, tsconfig.json, next.config.mjs, postcss.config.mjs
    `-- app/layout.tsx, globals.css, page.tsx,
        login/page.tsx, register/page.tsx, dashboard/page.tsx
```

Configuration is supplied through environment files. The backend reads `DATABASE_URL` (a go-sql-driver DSN with `parseTime=true` and `charset=utf8mb4`), `PORT` (8080), `FRONTEND_ORIGIN` (`http://localhost:3000`, used for CORS) and `COOKIE_SECURE` (false locally, true in production HTTPS). The frontend reads `NEXT_PUBLIC_API_URL = http://localhost:8080/api/v1`. Real credentials are kept only in the gitignored `.env` and `.env.local` files, while the `.env.example` files document the expected variables.

4.2 Tools and Technologies Used

Table 4.1: Tools and technologies

Layer	Technology	Role in TaskFlow
Backend language	Go 1.22+ (run on Go 1.27)	Compiled, concurrent language for the REST API
HTTP server	`net/http` (standard library)	HTTP server with configured timeouts

Layer	Technology	Role in TaskFlow
Routing	julienschmidt/httprouter v1.3.0	Radix-tree request multiplexer with path parameters
Database access	`database/sql` + go-sql-driver/mysql v1.8.1	Connection pooling and MySQL driver
Password security	golang.org/x/crypto/bcrypt v0.27.0	Salted password hashing (default cost)
Token security	`crypto/rand`, `crypto/sha256`	Random session tokens and their hashes
Frontend framework	Next.js 16.4.3 (App Router, Turbopack)	Routing, rendering and bundling
UI library	React 19, React-DOM 19	Component-based interface and state
Language	TypeScript (strict)	Static typing of frontend code
Styling	Tailwind CSS, PostCSS, Autoprefixer	Utility classes and custom design system
Icons / feedback	lucide-react, Sonner	SVG icons and toast notifications
Forms (available)	react-hook-form, zod, @hookform/resolvers	Form and schema validation libraries included in dependencies
Database	MySQL 8.0, InnoDB, utf8mb4	Persistent relational storage

4.3 Backend Implementation

The backend has two entry points. `cmd/migrate/main.go` is a standalone, idempotent migration runner that creates the `taskflow` database and executes the files in `migrations/` in numerical order; because it is idempotent it can be run repeatedly without damaging an existing schema. `cmd/api/main.go` starts the API server: it loads configuration, opens the MySQL connection pool, builds the router and listens on the configured port.

When a request arrives, the processing follows a fixed lifecycle. (1) The CORS wrapper checks the `Origin` header against `FRONTEND_ORIGIN`, adds `Access-Control-Allow-Credentials: true` and answers preflight `OPTIONS` requests with `204 No Content`. (2) httprouter matches the method and path and extracts parameters such as `:id`. (3) For protected routes the authentication middleware reads the `taskflow_session` cookie, computes its SHA-256 hash and resolves it to a user through the sessions table. (4) The handler decodes and validates the JSON body or query parameters and calls a repository method with the authenticated user ID. (5) The repository executes a parameterised statement and maps rows to model structs. (6) The handler encodes the result as JSON inside a `data` object with the appropriate status code.

Validation is applied at more than one level. The frontend checks email format, password length and confirmation before sending a request; the handlers validate request fields such as the task title, status, priority, sort order and the pagination limit (maximum 100); and the database enforces NOT NULL, UNIQUE, CHECK and FOREIGN KEY constraints as a final safeguard. The specification documents the success responses of each endpoint; specific error status codes and error-body formats are not documented and are therefore not reproduced in this report.

Data access and per-user isolation

Every task query issued by the repository contains the authenticated user's ID as a bound parameter, for example `WHERE id = ? AND user_id = ?` for single-task operations. A request for a task that belongs to another user therefore matches no row, so users can neither read nor modify each other's data. The listing query combines optional conditions for status, priority and a text search over title and description with one of the documented sort orders and the limit and offset values, and returns a pagination object containing `limit`, `offset` and `total`.

4.4 REST API Implementation

All endpoints share the base URL `http://localhost:8080/api/v1`, accept and return JSON, and use cookie authentication. Table 4.2 lists the twelve documented endpoints.

Table 4.2: REST API reference

Method	Endpoint	Auth	Purpose / success response
GET	`/health`	No	Health check: 200 `{"status":"ok"}`
POST	`/auth/register`	No	Create account; 201 with user; sets session cookie
POST	`/auth/login`	No	Verify credentials; 200 with user; sets session cookie
GET	`/auth/me`	Yes	Current user profile (id, name, email, created_at)
POST	`/auth/logout`	Yes	End session; 200 `logged_out`; cookie MaxAge = -1
GET	`/tasks`	Yes	List tasks with search, filter, sort and pagination
GET	`/tasks/stats`	Yes	Aggregated counts and completion rate
POST	`/tasks`	Yes	Create task; 201 with task object
GET	`/tasks/:id`	Yes	Retrieve one task
PUT	`/tasks/:id`	Yes	Update task fields; 200 with updated task
PATCH	`/tasks/:id/status`	Yes	Set status; 200 with new status
DELETE	`/tasks/:id`	Yes	Delete task; 200 `deleted`

Table 4.3: Query parameters of GET /tasks

Parameter	Allowed values	Default
`status`	pending | completed	all statuses
`priority`	low | medium | high	all priorities
`search`	text matched against title or description	none
`sort`	created_desc | created_asc | due_date_asc | due_date_desc | priority_desc | title_asc	not specified
`limit`	items per page, maximum 100	50
`offset`	pagination offset	0

The following request and responses are the examples given in the project specification for task creation and statistics (the example password value in the specification is omitted here):

```
POST /api/v1/tasks                          (cookie: taskflow_session)
{ "title": "Build API",
  "description": "Implement authentication endpoints",
  "priority": "high", "due_date": "2026-11-01" }
-> 201 Created   {"data": { ...task object... }}

GET /api/v1/tasks/stats
-> 200 OK  {"data": {"total": 10, "pending": 4, "completed": 6,
                     "high_priority": 2, "overdue": 1, "completion_rate": 60.0}}
```

4.5 Frontend Implementation

The frontend uses the Next.js App Router, in which each folder under `app/` defines a route through its `page.tsx` file. `layout.tsx` renders the root HTML structure and the Sonner `Toaster`, so any page can display notifications. The login, registration and dashboard pages are interactive Client Components that keep form and view state in React and call the API asynchronously using `NEXT_PUBLIC_API_URL`. Requests are made with credentials enabled so that the browser stores the HttpOnly cookie set by the API and sends it with later requests; the frontend code never reads the token itself.

On the dashboard, user actions are translated into API calls. Changing a status pill, the priority or sort dropdown, or the search text issues `GET /tasks` with the corresponding query parameters; the check button sends `PATCH /tasks/:id/status`; the edit button opens the modal pre-filled with the task and saves with `PUT /tasks/:id`; the delete button asks for confirmation before

sending `DELETE /tasks/:id`; and the statistics cards are populated from `GET /tasks/stats`. Overdue status is shown for pending tasks whose due date is earlier than the current date, and the completion bar is drawn from the completion rate.

4.6 Security Implementation

Table 4.4: Security mechanisms

Threat / concern	Mechanism implemented in TaskFlow
Password disclosure	bcrypt with automatic salt and default cost; plaintext passwords are never logged or stored
Session theft from the database	Only the SHA-256 hash of the token is stored; the raw 64-hex-character token exists only in the cookie
Token guessing	32 cryptographically secure random bytes from `crypto/rand`
XSS token theft	Cookie flag `HttpOnly` hides the token from JavaScript
CSRF	Cookie flag `SameSite=Lax`; exact-origin CORS
Interception	`Secure` flag enabled through `COOKIE_SECURE` for production HTTPS
SQL injection	Prepared statements with `?` placeholders; user input is never concatenated into SQL
Cross-user access	Every task query scoped with `user_id` of the authenticated session
Stale sessions	Seven-day cookie expiry; `expires_at` stored and indexed for cleanup

4.7 Setup and Execution

The project is run locally in the following order, as documented in the specification:

```
1. Start MySQL Server 8.0 on port 3306 and set credentials in backend/.env
2. cd backend
   go run ./cmd/migrate      # creates database 'taskflow' and runs
migrations
3. go run ./cmd/api          # API listens on http://localhost:8080
4. cd ../frontend
   npm install
   npm run dev               # Next.js starts on http://localhost:3000
5. Open http://localhost:3000 in a web browser
```

OUTPUT

5.1 Public and Authentication Screens

5.1: Landing Page

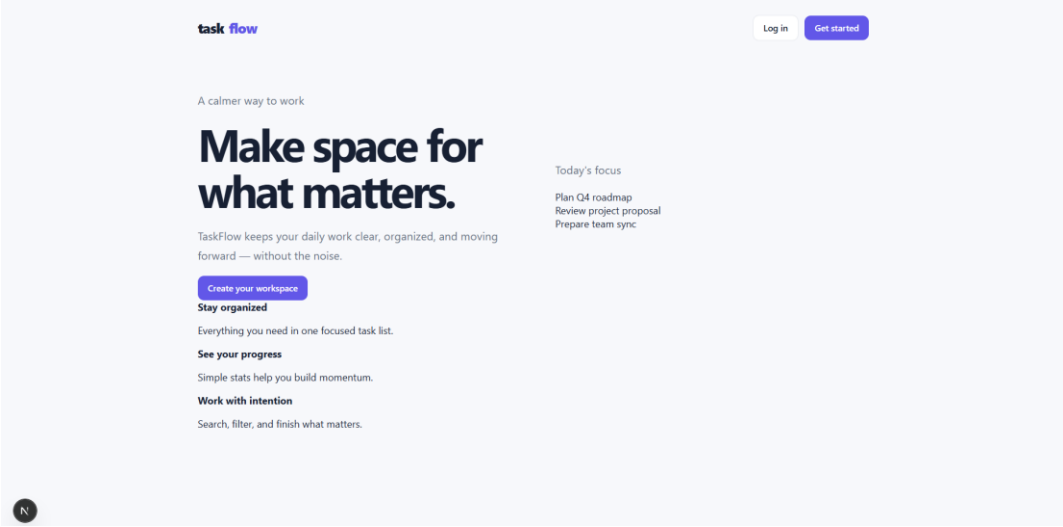


Figure 5.1: TaskFlow landing page

5.2: Registration Page

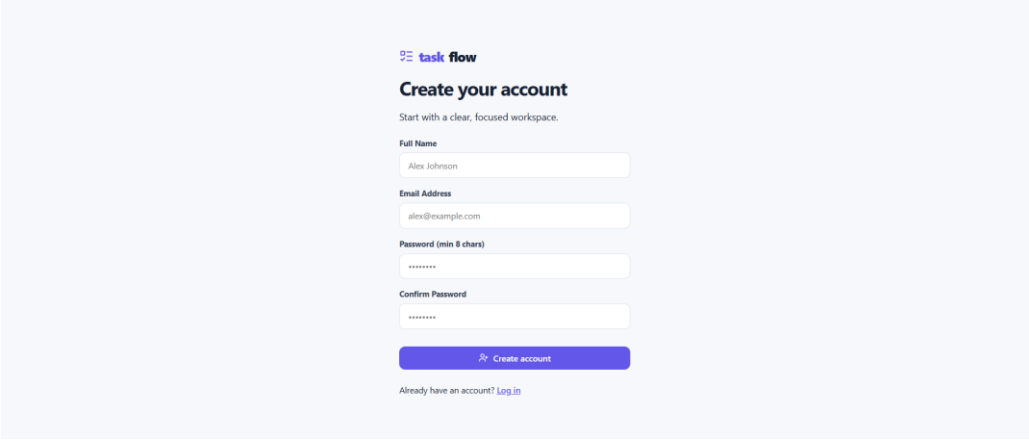


Figure 5.2: User registration page

5.3: Login Page

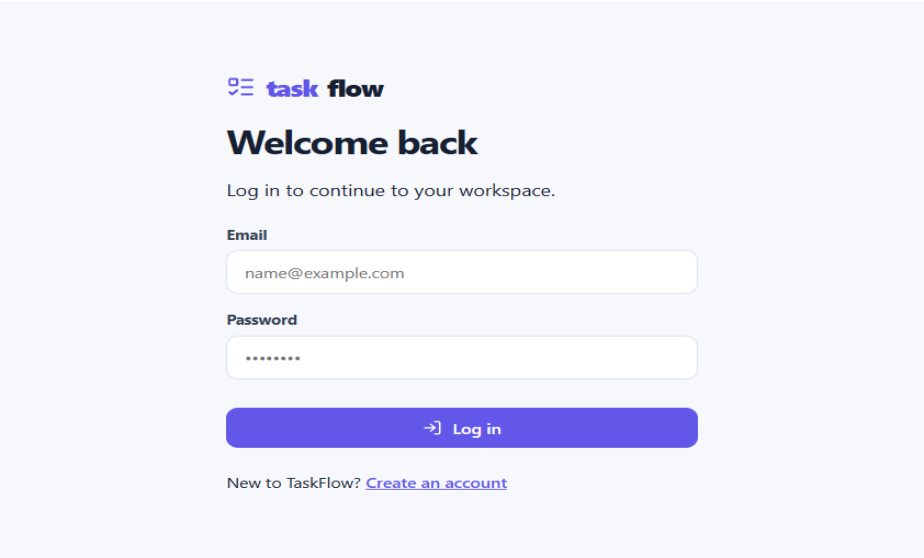


Figure 5.3: User login page

5.2 Dashboard and Task Management Screens

5.4: Dashboard Overview

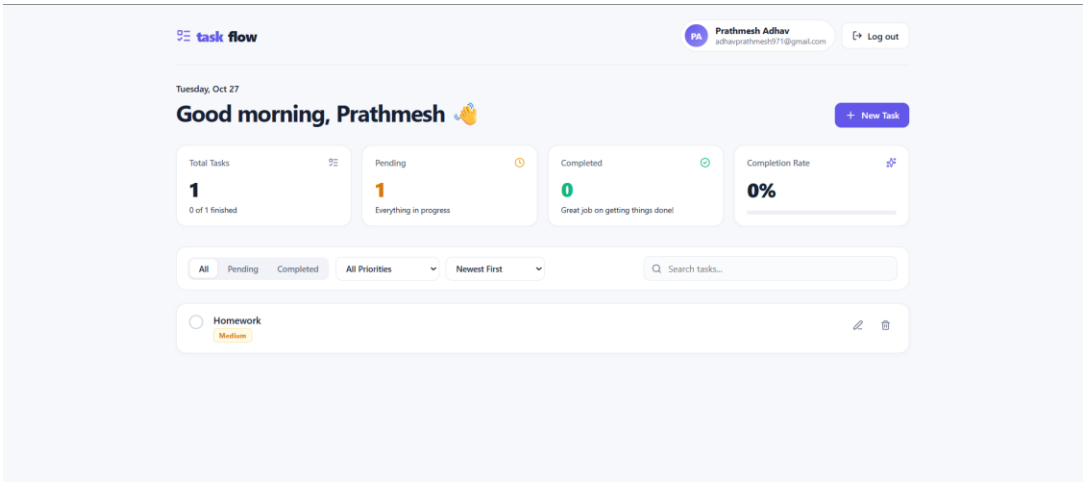


Figure 5.4: Dashboard with profile and task statistics

Screenshot 5.5: Create Task Modal

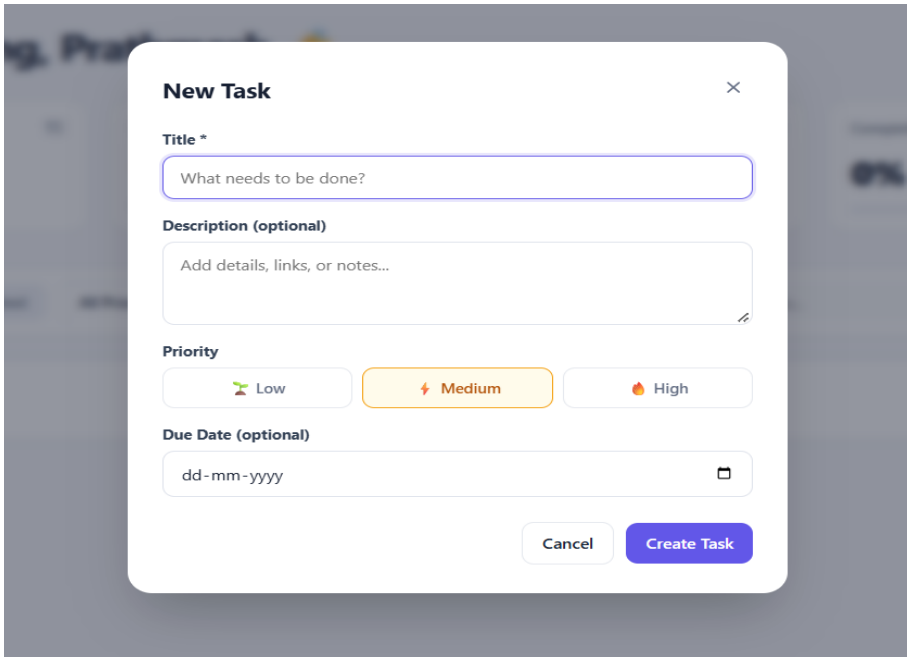


Figure 5.5: Create task modal dialog

Screenshot 5.6: Task List with Priorities and Overdue Badge

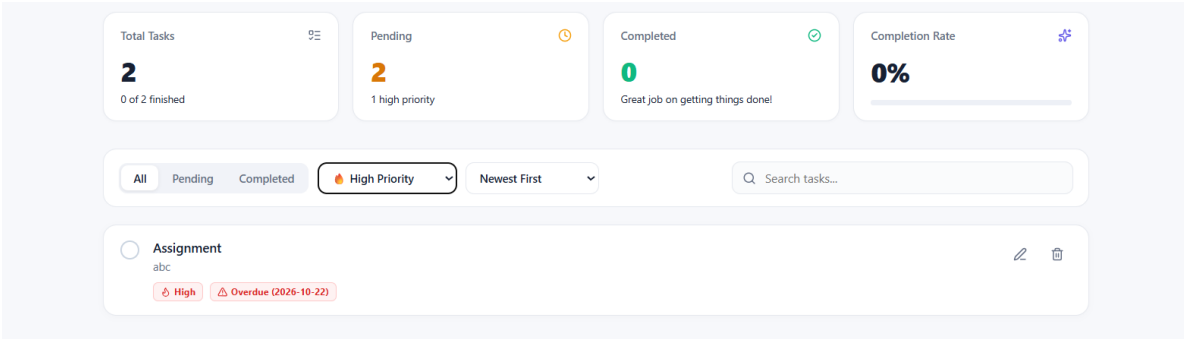


Figure 5.6: Task list showing priorities, completion and overdue detection

Screenshot 5.7: Search, Filter and Sort

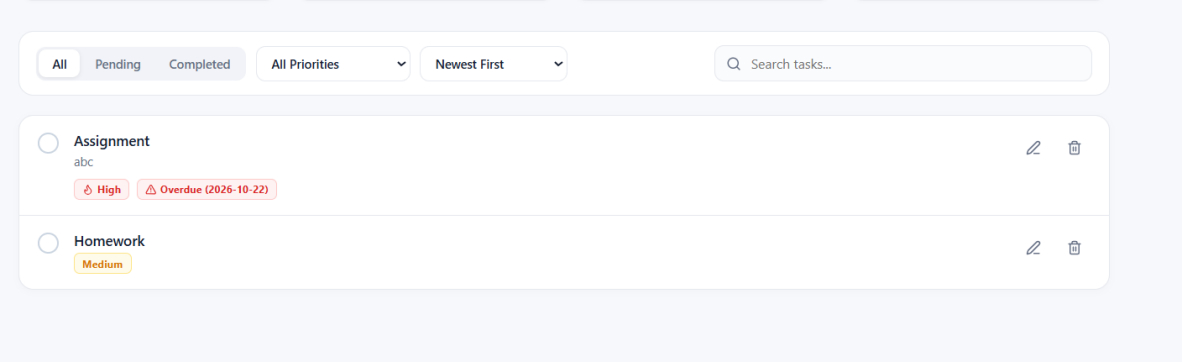


Figure 5.7: Search, filtering and sorting of tasks

Screenshot 5.8: Edit and Delete Task

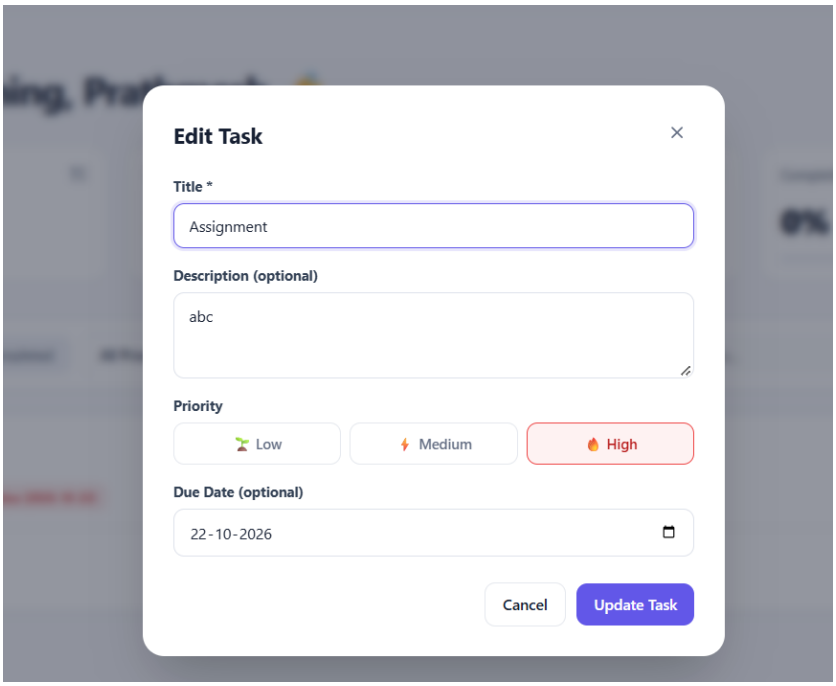


Figure 5.8: Editing and deleting a task

ADVANTAGES & LIMITATIONS

6.1 Advantages

- **Strong security by design:** bcrypt password hashes, hashed session tokens, HttpOnly and SameSite cookies, exact-origin CORS and parameterised SQL address the most common web-application vulnerabilities.
- **Strict data isolation:** scoping every task query by `user_id` ensures that each user sees and changes only their own tasks.
- **Clean, maintainable architecture:** the decoupled frontend and API and the layered router-handler-repository backend allow each part to be developed and tested independently.
- **Reliable data integrity:** foreign keys with cascade delete, unique constraints and check constraints keep the database consistent even if invalid input reaches it.
- **Query efficiency:** composite indexes that start with `user_id` match the dashboard's filtering and sorting patterns, and connection pooling reuses database connections.
- **Productive user experience:** statistics, a completion bar, overdue badges, instant search and filtering, colour-coded priorities and modal editing make task tracking quick.
- **Repeatable setup:** environment files and idempotent, ordered migrations allow the project to be installed consistently on a new machine.
- **Lightweight backend:** the API relies mainly on Go's standard library with a small number of focused dependencies.

6.2 Limitations

- **Individual scope:** tasks belong to a single account; sharing tasks, team workspaces and role-based permissions are not implemented.
- **Local deployment:** the documented configuration targets localhost; production hosting, HTTPS certificates and deployment automation are not described.

- **Limited task model:** tasks have one status pair (pending/completed) and no categories, tags, sub-tasks, attachments or reminders.
- **No notifications:** overdue tasks are highlighted only when the dashboard is opened; no email or push alerts are sent.
- **Account management:** password reset, email verification and profile editing are not part of the documented endpoints.
- **Testing and performance evidence:** the specification does not document automated tests or load measurements, so performance and scalability under heavy use have not been verified.
- **Session cleanup:** an index on `expires_at` supports cleanup of expired sessions, but a scheduled cleanup job is not described.

6.3 Future Enhancements

The following improvements are proposals and are not part of the current implementation: shared projects and team task assignment with role-based access; email or browser notifications for approaching and overdue deadlines; password reset and email verification; tags, categories and sub-tasks; automated unit, integration and end-to-end tests; containerised deployment with HTTPS and a managed MySQL instance; and a scheduled job that deletes expired sessions using the existing `idx_sessions_expiry` index.

CONCLUSION

TaskFlow demonstrates the design and implementation of a complete, secure web application using current web technologies. A Next.js 16 and React 19 frontend written in TypeScript provides a responsive dashboard, while a layered Go REST API and a normalised MySQL schema provide reliable, user-isolated storage of tasks.

7.1 Summary of the Project Achievements

- Implemented registration, login, logout and profile retrieval with bcrypt-hashed passwords and SHA-256-hashed, cookie-based sessions.
- Implemented the full task lifecycle with priorities, due dates and overdue detection, plus search, filtering, six sort orders and pagination.
- Built a dashboard with statistics cards, a completion-rate progress bar, colour-coded priority badges and modal editing.
- Designed three InnoDB tables with primary keys, foreign keys with cascade delete, unique and check constraints and query-oriented composite indexes, created through ordered migrations.
- Applied credentialed CORS, parameterised SQL and per-user query scoping to protect data.

7.2 Key Learning

Key learning outcomes include designing RESTful endpoints with suitable HTTP methods and status codes; implementing cookie-based session authentication with HttpOnly, SameSite and Secure attributes; configuring CORS for credentialed cross-origin requests; structuring a Go backend into router, handler, model and repository layers; designing a relational schema with keys, constraints and indexes that match application queries; preventing SQL injection with prepared statements; and building typed, component-based user interfaces with Next.js, React and Tailwind CSS.

REFERENCES

8.1 Books, Articles, Websites and Other Resources

[1] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[2] R. Fielding, M. Nottingham and J. Reschke, "HTTP Semantics," RFC 9110, IETF, June 2022. https://www.rfc-editor.org/rfc/rfc9110

[3] A. Barth, "HTTP State Management Mechanism," RFC 6265, IETF, April 2011. https://www.rfc-editor.org/rfc/rfc6265

[4] The Go Authors, "Package net/http," Go Documentation. https://pkg.go.dev/net/http

[5] The Go Authors, "Package database/sql," Go Documentation. https://pkg.go.dev/database/sql

[6] Go-MySQL-Driver contributors, "Go-MySQL-Driver." https://github.com/go-sql-driver/mysql

[7] J. Schmidt, "HttpRouter." https://github.com/julienschmidt/httprouter

[8] The Go Authors, "Package bcrypt," golang.org/x/crypto. https://pkg.go.dev/golang.org/x/crypto/bcrypt

[9] Vercel, "Next.js Documentation: App Router." https://nextjs.org/docs/app

[10] Meta Open Source, "React Documentation." https://react.dev

[11] Microsoft, "TypeScript Documentation." https://www.typescriptlang.org/docs/

[12] Tailwind Labs, "Tailwind CSS Documentation." https://tailwindcss.com/docs

[13] Oracle Corporation, "MySQL 8.0 Reference Manual." https://dev.mysql.com/doc/refman/8.0/en/

[14] Mozilla, "Using HTTP cookies," MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies

[15] Mozilla, "Cross-Origin Resource Sharing (CORS)," MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS

[16] OWASP Foundation, "SQL Injection Prevention Cheat Sheet." https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

[17] OWASP Foundation, "Session Management Cheat Sheet." https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[18] OWASP Foundation, "Password Storage Cheat Sheet." https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html